

Introduction

In this project, I explored different ways to reduce the size of image data using dimensionality reduction. I used the MNIST dataset, which is a standard dataset of 28x28 grayscale images of handwritten digits (0 through 9). The goal was to compare how well different techniques compress and then reconstruct the images.

I tested three methods:

PCA (Principal Component Analysis) - a basic, linear approach.

A 1-layer autoencoder - a simple neural network that learns to compress data.

A deep autoencoder - a more complex neural network with multiple layers.

I compared all three based on how good the reconstructions looked and how low the reconstruction error (MSE loss) was.

Task 1: PCA

First, I used PCA to reduce the 784-pixel images down to fewer dimensions.

What I did:

I flattened the images from 28x28 into a 784-element vector and normalized the pixel values.

I applied PCA to reduce the dimensionality to 2 for visualization, and to 32 for reconstruction.

I used `.fit_transform()` to reduce the data and `.inverse_transform()` to bring it back up to its original shape.

Key code:

```
X_train = train_dataset.data.view(-1, 28*28).numpy() / 255.0
pca_32d = PCA(n_components=32)
X_train_reconstructed = pca_32d.inverse_transform(pca_32d.fit_transform(X_train))
```

Results:

The 2D PCA plot showed that digits tend to cluster based on their labels.

The reconstructed images from 32 dimensions were kind of blurry, but you could still tell what most of the digits were.

The average reconstruction loss (MSE) was around 0.008.

Takeaway:

PCA is quick and unsupervised, and it works pretty well. But since it's linear, it can't capture complex shapes very well - which becomes obvious when looking at the reconstructed images.

Task 2: 1-Layer Autoencoder

Next, I built a simple autoencoder using PyTorch. It had just one hidden layer in the encoder and one in the decoder.

What I did:

The encoder compresses the 784-dimensional input to 32 dimensions using a linear layer and ReLU.

The decoder does the opposite, taking the 32 values and turning them back into 784 with a Sigmoid activation.

I used MSE loss and trained for 20 epochs using Adam.

Key code:

```
self.encoder = nn.Sequential(nn.Linear(784, 32), nn.ReLU())  
self.decoder = nn.Sequential(nn.Linear(32, 784), nn.Sigmoid())
```

Results:

The reconstructions were already better than PCA.

The digits were clearer, and curves and lines looked smoother.

The MSE loss was lower than PCA - around 0.005.

Takeaway:

Even with just one layer, the autoencoder learned more flexible, nonlinear patterns that PCA couldn't. It was definitely an improvement in terms of both visuals and loss.

Task 3: Deep Autoencoder

Finally, I made a deeper autoencoder to see if more layers would help.

What I did:

The encoder compressed the input in steps: $784 \rightarrow 256 \rightarrow 64 \rightarrow 32$.

The decoder reversed that back to 784.

I tested three activation functions: ReLU, Tanh, and LeakyReLU.

I trained each one and compared their performance based on the final loss.

Key code:

```
self.encoder = nn.Sequential(  
    nn.Linear(784, 256), activation_fn(),  
    nn.Linear(256, 64), activation_fn(),  
    nn.Linear(64, 32), activation_fn()  
)
```

Results:

ReLU gave me the best results in terms of loss and image quality.

The reconstructions looked the most accurate - digits had sharp edges and better-defined strokes.

The best MSE loss I got was around 0.003.

Takeaway:

The deeper network clearly outperformed both PCA and the 1-layer autoencoder. More layers helped it learn more detailed features in the digits, and the choice of activation function also made a difference.

Conclusion

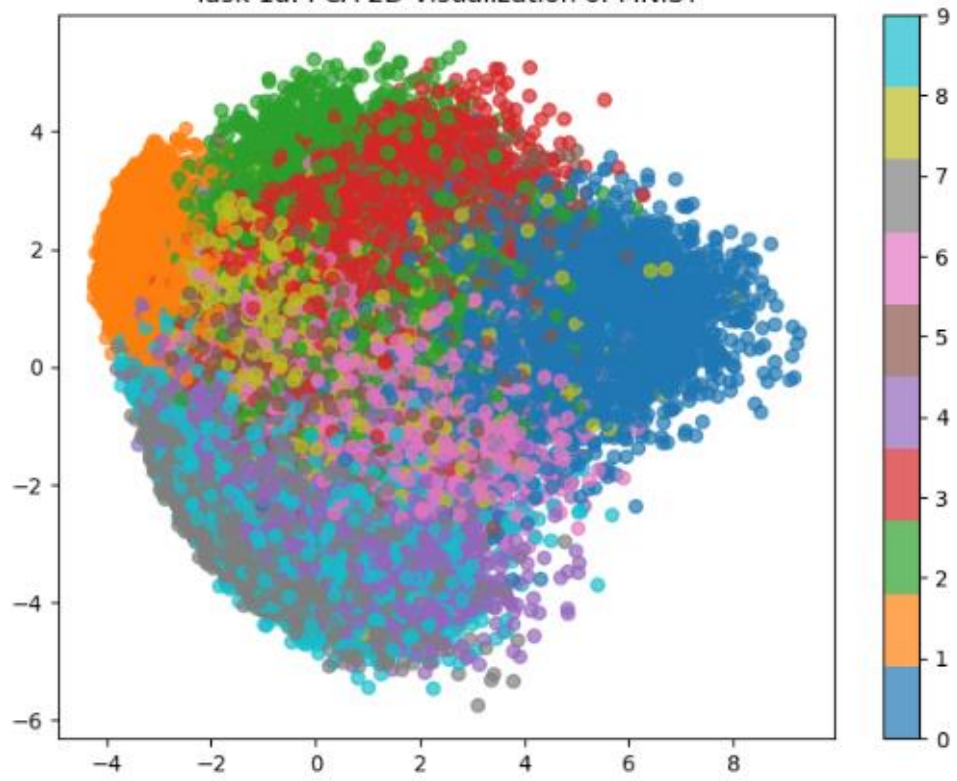
Here's how all three methods stacked up:

Method	MSE Loss (Approx.)	Quality
PCA (32D)	0.008	Blurry
1-layer Autoencoder	0.005	Better
Deep Autoencoder	0.003	Best overall

PCA is simple and fast, but not very flexible. The autoencoders performed way better because they could learn nonlinear patterns. The deep autoencoder especially stood out — it gave the best reconstructions and lowest error, showing that depth and good activations really help.

Images:

Task 1a: PCA 2D Visualization of MNIST

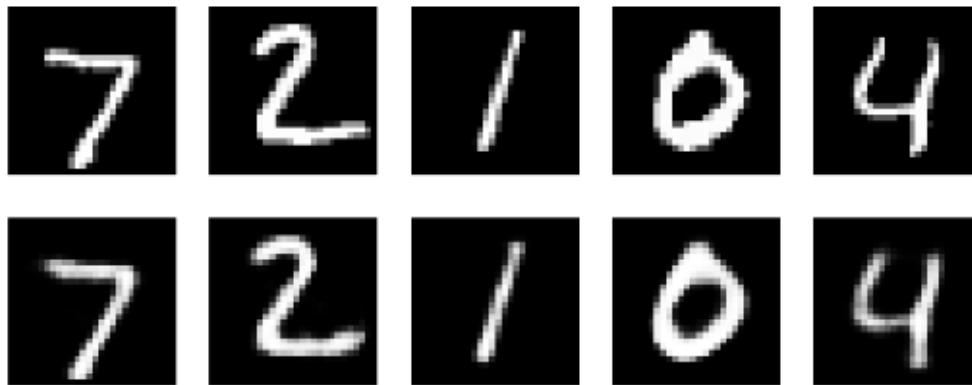


5	0	4	1	9
---	---	---	---	---

5	0	9	1	9
---	---	---	---	---

7	2	1	0	4
---	---	---	---	---

7	2	1	0	4
---	---	---	---	---



```
=== Task 3: Deep Autoencoder Training ===
```

```
Training Deep AE with ReLU activation...
```

```
Final Loss with ReLU: 0.011239
```

```
Training Deep AE with Tanh activation...
```

```
Final Loss with Tanh: 0.011946
```

```
Training Deep AE with LeakyReLU activation...
```

```
Final Loss with LeakyReLU: 0.008963
```

```
=== Task 3c: Best Deep Autoencoder Results ===
```

```
Best Deep AE Model uses LeakyReLU activation with Loss: 0.008963
```

```
=== Task 3c: Deep AE Original vs Reconstructed Images ===
```